

SAEPBiblioteca

Sistema de Gerenciamento de Biblioteca

Aplicação Spring Boot + MySQL + SPA (HTML/CSS/JS) para gerenciamento de livros, membros, empréstimos e avaliações.

Manual de Construção — Versão 1.0

Stack: Spring Boot 3.2.3 + MySQL 8 + HTML/CSS/JS (SPA)

Etapa 0 — Modelagem do Banco de Dados

0.1 Diagrama Conceitual

A aplicacao **SAEPBiblioteca** possui 4 tabelas principais: `tb_membros` , `tb_livros` , `tb_favoritos` e `tb_resenhas` .

Tabela	Descricao	Relacionamentos
<code>tb_membros</code>	Cadastro de membros do sistema	1 : N com <code>tb_livros</code> , <code>tb_favoritos</code> , <code>tb_resenhas</code>
<code>tb_livros</code>	Registros principais do dominio	N : 1 com <code>tb_membros</code> , 1 : N com <code>tb_favoritos</code> e <code>tb_resenhas</code>
<code>tb_favoritos</code>	Reacoes/interacoes dos membros	N : 1 com <code>tb_membros</code> e <code>tb_livros</code>
<code>tb_resenhas</code>	Comentarios/observacoes	N : 1 com <code>tb_membros</code> e <code>tb_livros</code>

Etapa 1 — Setup do Projeto Spring Boot + MySQL

1.1 Criar projeto via Spring Initializr

Dependências: Spring Web, Spring Data JPA, MySQL Driver, Lombok.

1.2 Configurar `pom.xml`

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.3</version>
</parent>
<dependencies>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId></dependency>
  <dependency><groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId><scope>runtime</scope></dependency>
  <dependency><groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId><optional>true</optional></dependency>
</dependencies>
```

1.3 Configurar o MySQL — `application.properties`

```
spring.application.name=sapbiblioteca
spring.datasource.url=jdbc:mysql://localhost:3306/sapbiblioteca?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=037397
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
spring.sql.init.mode=always
spring.jpa.defer-datasource-initialization=true
```

Checkpoint Etapa 1: A aplicação sobe sem erros em `http://localhost:8080` e o banco `sapbiblioteca` é criado.

Etapa 2 — Criar as Entidades (Model)

2.1 Membro.java

```
package com.saep.saepbiblioteca.model;
import jakarta.persistence.*;
import lombok.Data;
import lombok.NoArgsConstructor;
import java.time.LocalDateTime;

@Data @NoArgsConstructor @Entity
@Table(name = "tb_membros")
public class Membro {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;
    private String email;
    private String nomeUsuario;
    private String imagem;
    private String senha;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
}
```

2.2 Livro.java

```
package com.saep.saepbiblioteca.model;
import jakarta.persistence.*;
import lombok.Data; import lombok.NoArgsConstructor;
import java.time.LocalDateTime; import java.util.List;

@Data @NoArgsConstructor @Entity
@Table(name = "tb_livros")
public class Livro {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String titulo;
    private String autor;
    private String genero;
    private Integer anoPublicacao;
    private Integer numeroPaginas;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
    @ManyToOne @JoinColumn(name = "membro_id")
    private Membro membro;
    @OneToMany(mappedBy = "livro", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Favorito> favoritos;
    @OneToMany(mappedBy = "livro", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Resenha> resenhas;
}
```

2.3 Favorito.java

```
package com.saep.saepbiblioteca.model;
import jakarta.persistence.*;
import lombok.Data; import lombok.NoArgsConstructor;
import com.fasterxml.jackson.annotation.JsonIgnore;

@Data @NoArgsConstructor @Entity
@Table(name = "tb_favoritos")
public class Favorito {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @ManyToOne @JoinColumn(name = "membro_id")
    private Membro membro;
```

```

@ManyToOne @JoinColumn(name = "livro_id")
@JsonIgnore // evita loop infinito na serializacao JSON
private Livro livro;
}

```

2.4 Resenha.java

```

package com.saep.saeplibiblioteca.model;
import jakarta.persistence.*;
import lombok.Data; import lombok.NoArgsConstructor;
import com.fasterxml.jackson.annotation.JsonIgnore;

@Data @NoArgsConstructor @Entity
@Table(name = "tb_resenhas")
public class Resenha {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String texto;
    @ManyToOne @JoinColumn(name = "membro_id")
    private Membro membro;
    @ManyToOne @JoinColumn(name = "livro_id")
    @JsonIgnore
    private Livro livro;
}

```

Checkpoint Etapa 2: Ao subir a aplicacao, as 4 tabelas sao criadas no MySQL (verifique com `SHOW TABLES;`).

Etapa 3 — Criar os Repositories

3.1 MembroRepository.java

```
package com.saep.saepbiblioteca.repository;
import com.saep.saepbiblioteca.model.Membro;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface MembroRepository extends JpaRepository<Membro, Long> {
    Optional<Membro> findByEmailAndSenha(String email, String senha);
}
```

3.2 LivroRepository.java

```
package com.saep.saepbiblioteca.repository;
import com.saep.saepbiblioteca.model.Livro;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface LivroRepository extends JpaRepository<Livro, Long> {
    @Query("SELECT a FROM Livro a WHERE :tipo IS NULL OR a.genero = :tipo ORDER BY a.createdAt DESC")
    Page<Livro> findByGenero(@Param("tipo") String tipo, Pageable pageable);

    long countByMembroId(Long membroId);

    @Query("SELECT COALESCE(SUM(a.numeroPaginas), 0) FROM Livro a WHERE a.membro.id = :id")
    long sumPaginasByMembroId(@Param("id") Long id);
}
```

3.3 FavoritoRepository.java

```
public interface FavoritoRepository extends JpaRepository<Favorito, Long> {
    Optional<Favorito> findByMembroIdAndLivroId(Long membroId, Long livroId);
}
```

3.4 ResenhaRepository.java

```
public interface ResenhaRepository extends JpaRepository<Resenha, Long> { }
```

Checkpoint Etapa 3: O código compila sem erros (`mvn compile`).

Etapa 4 — Importar Dados do Prototipo (SQL)

PARTE PRINCIPAL: Os dados devem ser importados via `data.sql` para popular o banco.

4.1 `data.sql` — `src/main/resources/data.sql`

4.1.1 Inserts para `tb_membros`

```
INSERT IGNORE INTO tb_membros (id, nome, email, nome_usuario, imagem, senha, created_at, updated_at) VALUES
('1', 'saeplivros', 'saeplivros@email.com', 'saeplivros', 'saeplivros.png', '123456', '2024-08-14 18:56:33', '2024-08-14 18:56:33'),
('2', 'leitor1', 'leitor1@email.com', 'leitor01', 'leitor01.jpg', '123456', '2024-08-14 18:58:07', '2024-08-14 18:58:07'),
('3', 'leitor2', 'leitor2@email.com', 'leitor02', 'leitor02.jpg', '123456', '2024-08-14 18:58:21', '2024-08-14 18:58:21'),
('4', 'leitor3', 'leitor3@email.com', 'leitor03', 'leitor03.jpg', '123456', '2024-08-14 18:58:35', '2024-08-14 18:58:35');
```

4.1.2 Inserts para `tb_livros`

```
INSERT IGNORE INTO tb_livros (id, genero, titulo, numero_paginas, ano_publicacao, created_at, updated_at, membro_id) VALUES
(3, 'Ficcao', 'Dom Casmurro', 256, 1899, '2024-08-14 19:15:11', '2024-08-14 19:15:11', 1),
(4, 'Ficcao', 'Memorias Postumas', 342, 1881, '2024-08-14 19:15:54', '2024-08-14 19:15:54', 2),
(5, 'Ficcao', 'O Alienista', 168, 1882, '2024-08-14 19:16:09', '2024-08-14 19:16:09', 3),
(6, 'Ficcao', 'Quincas Borba', 320, 1891, '2024-08-14 19:16:24', '2024-08-14 19:16:24', 4),
(7, 'Nao-Ficcao', 'O Povo Brasileiro', 480, 1995, '2024-08-14 19:17:30', '2024-08-14 19:17:30', 1),
(8, 'Nao-Ficcao', 'Casa Grande & Senzala', 640, 1933, '2024-08-14 19:17:47', '2024-08-14 19:17:47', 2),
(9, 'Nao-Ficcao', 'Raizes do Brasil', 256, 1936, '2024-08-14 19:18:18', '2024-08-14 19:18:18', 3),
(10, 'Nao-Ficcao', 'Historia do Brasil', 512, 1999, '2024-08-14 19:18:41', '2024-08-14 19:18:41', 4),
(11, 'Tecnico', 'Java Efetivo', 416, 2018, '2024-08-14 19:20:24', '2024-08-14 19:20:24', 1),
(12, 'Tecnico', 'Codigo Limpo', 464, 2008, '2024-08-14 19:20:43', '2024-08-14 19:20:43', 2),
(13, 'Tecnico', 'Domain-Driven Design', 560, 2003, '2024-08-14 19:26:04', '2024-08-14 19:26:04', 3),
(14, 'Tecnico', 'Padroes de Projeto', 416, 1994, '2024-08-14 19:26:23', '2024-08-14 19:26:23', 4);
```

IMPORTANTE: `INSERT IGNORE` evita duplicar dados ao reiniciar a aplicacao. Os IDs fixos garantem integridade das chaves estrangeiras.

Checkpoint Etapa 4: Após subir a aplicacao, `SELECT * FROM tb_membros;` e `SELECT * FROM tb_livros;` retornam os dados.

Etapa 5 — Criar a API REST (Controller)

5.1 ApiController.java

Login

```
@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody LoginRequest request) {
    Optional<Membro> membro =
        membroRepository.findByEmailAndSenha(request.getEmail(), request.getSenha());
    if (membro.isPresent())
        return ResponseEntity.ok(membro.get());
    return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Email ou senha incorreta");
}
```

Perfil — Totais

```
@GetMapping("/perfil/{usuarioId}")
public ResponseEntity<?> getPerfil(@PathVariable Long usuarioId) {
    long total = livroRepository.countByMembroId(usuarioId);
    long soma = livroRepository.sumPaginasByMembroId(usuarioId);
    Map<String, Object> response = new HashMap<>();
    response.put("totalAtividades", total);
    response.put("totalCalorias", soma);
    return ResponseEntity.ok(response);
}
```

Listar com Paginação e Filtro

```
@GetMapping("/livros")
public ResponseEntity<Page<Livro>> getItens(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(required = false) String tipo) {
    Pageable pageable = PageRequest.of(page, 4); // 4 por pagina
    return ResponseEntity.ok(livroRepository.findByGenero(tipo, pageable));
}
```

Criar

```
@PostMapping("/livros")
public ResponseEntity<?> criar(@RequestBody CriarRequest request) {
    Membro membro = membroRepository.findById(request.getUsuarioId()).orElse(null);
    if (membro == null)
        return ResponseEntity.badRequest().body("Membro nao encontrado");
    Livro item = new Livro();
    item.setTitulo(request.getTitulo());
    item.setAutor(request.getAutor());
    item.setGenero(request.getGenero());
    item.setAnoPublicacao(request.getAnoPublicacao());
    item.setCreatedAt(LocalDateTime.now());
    item.setUpdatedAt(LocalDateTime.now());
    item.setMembro(membro);
    livroRepository.save(item);
    return ResponseEntity.ok(item);
}
```

Reacao (Toggle)

```
@PostMapping("/livros/{id}/favoritar")
public ResponseEntity<?> reagir(@PathVariable Long id, @RequestBody InteracaoRequest request) {
```



```
Optional<Favorito> existe = favoritoRepository
    .findByMembroIdAndLivroId(request.getUsuarioId(), id);
if (existe.isPresent()) {
    favoritoRepository.delete(existe.get());
    return ResponseEntity.ok("Desfavoritou");
}
Favorito nova = new Favorito();
nova.setMembro(membroRepository.findById(request.getUsuarioId()).get());
nova.setLivro(livroRepository.findById(id).get());
favoritoRepository.save(nova);
return ResponseEntity.ok("Favoritou");
}
```

Comentario

```
@PostMapping("/livros/{id}/resenhar")
public ResponseEntity<> comentar(@PathVariable Long id, @RequestBody InteracaoRequest request) {
    if (request.getTexto() == null || request.getTexto().length() <= 2)
        return ResponseEntity.badRequest().body("nao e possivel enviar um comentario vazio");
    Resenha c = new Resenha();
    c.setMembro(membroRepository.findById(request.getUsuarioId()).get());
    c.setLivro(livroRepository.findById(id).get());
    c.setTexto(request.getTexto());
    resenhaRepository.save(c);
    return ResponseEntity.ok(c);
}
```

5.2 Mapa de Endpoints

Endpoint	Metodo	Descricao
/api/login	POST	Login do membro
/api/perfil/{id}	GET	Totais do membro
/api/livros?page=&genero=	GET	Listar com filtro
/api/livros	POST	Cadastrar livro
/api/livros/{id}/favoritar	POST	Favoritar/Desfavoritar
/api/livros/{id}/resenhar	POST	Enviar resenha

Checkpoint Etapa 5: Teste com Postman: GET `http://localhost:8080/api/livros?page=0` e POST `http://localhost:8080/api/login` com `{"email": "saeplivros@email.com", "senha": "123456"}` .

Etapa 6 — Estrutura HTML da SPA (Frontend)

PARTE PRINCIPAL: O Spring Boot serve arquivos estaticos da pasta `src/main/resources/static/` . A SPA nunca recarrega: tudo via `fetch()` .

6.1 `index.html`

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>SAEPBiblioteca</title>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600;700&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="livraria-app">

    <!-- COLUNA ESQUERDA: PERFIL -->
    <aside class="painel-esquerdo">
      <div class="cartao-leitor">
        
        <h2 id="nome-leitor">SAEPBiblioteca</h2>

        <div class="indicadores">
          <div class="metrica-box">
            <span id="total-livros" class="valor-metrica">0</span>
            <span class="descricao-metrica">Qtd. Livros Lidos</span>
          </div>
          <div class="metrica-box">
            <span id="total-paginas" class="valor-metrica">0</span>
            <span class="descricao-metrica">Qtd. Paginas</span>
          </div>
        </div>

        <button id="btn-adicionar-livro" class="btn-novo" disabled>
           Livro
        </button>
      </div>

      <footer class="rodape-painel">
        <p>SAEPBiblioteca</p>
        <div class="icones-sociais">
          
          
          
        </div>
        <p class="direitos-autorais">Copyright - 2024</p>
      </footer>
    </aside>

    <!-- COLUNA CENTRAL: CONTEUDO -->
    <main class="area-principal">
      <header class="topo-pagina">
        <button id="btn-sessao" class="btn-entrar">Login</button>
      </header>

      <div class="barra-generos">
        Ficcao
        Nao-Ficcao
        Tecnico
      </div>

      <div id="lista-atividades" class="grade-livros"></div>

      <div class="navegacao">
        <button id="btn-retroceder" class="btn-pagina">Anterior</button>
      </div>
    </main>
  </div>
</body>
</html>
```

```

        <div id="botoes-pagina" class="numeros-pagina"></div>
        <button id="btn-avancar" class="btn-pagina">Proxima</button>
    </div>
</main>
</div>

<!-- MODAL LOGIN -->
<div id="overlay-login" class="overlay">
    <div class="dialogo">
        <div class="topo-dialogo">
            <h2>Login</h2>
            
        </div>
        <form id="form-entrar">
            <div class="campo">
                <label for="campo-email">Email</label>
                <input type="email" id="campo-email">
            </div>
            <div class="campo">
                <label for="campo-senha">Senha</label>
                <input type="password" id="campo-senha">
            </div>
            <div class="botoes-dialogo">
                <button type="button" id="btn-cancelar-entrada" class="btn-desistir">Cancelar</button>
                <button type="submit" class="btn-confirmar">Login</button>
            </div>
        </form>
    </div>
</div>

```

```

<!-- MODAL CRIAR -->
<div id="overlay-criar" class="overlay">
    <div class="dialogo">
        <div class="topo-dialogo">
            <h2>Crie seu registro</h2>
            
        </div>
        <form id="form-novo-livro">
            <div class="linha-campos">
                <div class="campo meia-largura">
                    <label for="campo-genero">Tipo</label>
                    <select id="campo-genero" required>
                        <option value="">Selecione...</option>

```

Ficcao

Nao-Ficcao

Tecnico

```

                </select>
            </div>
            <div class="campo meia-largura">
                <label for="campo-ano">Numero de Paginas</label>
                <input type="number" id="campo-ano" placeholder="Ex: 300" required>
            </div>
        </div>
        <div class="linha-campos">
            <div class="campo meia-largura">
                <label for="campo-autor">Autor</label>
                <input type="text" id="campo-autor" placeholder="Ex: Machado de Assis">
            </div>
            <div class="campo meia-largura">
                <label for="campo-paginas">Ano de Publicacao</label>
                <input type="number" id="campo-paginas" placeholder="Ex: 1899" required>
            </div>
        </div>
        <div class="botoes-dialogo">
            <button type="submit" class="btn-confirmar">Criar Livro</button>
        </div>
    </form>
</div>
</div>

```

```

    <script src="app.js"></script>
</body>
</html>

```

6.2 Ativos estaticos para `static/assets/`

```
src/main/resources/static/assets/  
├─ logo_saepbiblioteca/ Logo.png  
├─ icones/ (coracao.svg, CoracaoVermelho.svg, comentario.svg, send.svg, x.svg, ...)  
├─ imagens_perfil/ (leitor01.jpg, leitor02.jpg, leitor03.jpg)  
└─ ...
```

Checkpoint Etapa 6: Acessar `http://localhost:8080` mostra o layout (sidebar escura + area principal), mesmo sem dados.

Etapa 7 — Folha de Estilos CSS

PARTE PRINCIPAL: Estrutura organizada com cores: #2c3e50 (fundo sidebar), #1a5276 (destaque), #FFFFFF (fontes).

7.1 style.css

```
/* =====
SAEPBiblioteca - Folha de Estilos
Fonte: Inter | Cores: #2c3e50 #FFFFFF #1a5276 #e74c3c
===== */

/* --- RESET --- */
* { margin: 0; padding: 0; box-sizing: border-box; font-family: 'Inter', sans-serif; }
body { background-color: #f5f5f5; }

/* --- LAYOUT GERAL --- */
.livraria-app { display: flex; min-height: 100vh; }

/* --- BARRA LATERAL - fundo #2c3e50 --- */
.painel-esquerdo {
  width: 300px;
  background-color: #2c3e50;
  color: #FFFFFF;
  display: flex;
  flex-direction: column;
  justify-content: space-between;
}

.cartao-leitor { padding: 40px 20px; text-align: center; }

.avatar-leitor {
  width: 120px; height: 120px; border-radius: 50%;
  margin-bottom: 20px; object-fit: cover;
  border: 3px solid #1a5276;
}

/* --- ESTATISTICAS --- */
.indicadores { display: flex; justify-content: space-around; margin: 30px 0; }
.metrica-box { display: flex; flex-direction: column; }
.valor-metrica { font-size: 24px; font-weight: bold; }
.descricao-metrica { font-size: 12px; color: #cccccc; }

/* --- BOTAO CRIAR --- */
.btn-novo {
  width: 100%; padding: 15px; background: transparent;
  border: 1px solid #FFFFFF; color: #FFFFFF;
  cursor: pointer; display: flex; align-items: center; justify-content: center;
  gap: 10px; font-size: 16px;
  transition: background-color 0.3s, border-color 0.3s;
}
.btn-novo:hover:not(:disabled) { background-color: #1a5276; border-color: #1a5276; }
.btn-novo:disabled { opacity: 0.5; cursor: not-allowed; }

/* --- RODAPE --- */
.rodape-painel { text-align: center; padding: 20px; border-top: 1px solid #444; font-size: 12px; color: #999; }

/* --- CONTEUDO PRINCIPAL --- */
.area-principal { flex: 1; display: flex; flex-direction: column; background-color: #f9f9f9; }
.topo-pagina { display: flex; justify-content: flex-end; padding: 20px; }

/* --- BOTÕES AUTENTICACAO --- */
.btn-entrar {
  padding: 10px 30px; background-color: #FFFFFF;
  color: #2c3e50; border: 1px solid #2c3e50;
  border-radius: 4px; font-weight: bold; cursor: pointer;
}
.btn-sair {
  padding: 10px 30px; background-color: #2c3e50;
  color: #FFFFFF; border: none;
  border-radius: 4px; font-weight: bold; cursor: pointer;
}
```

```

}

/* --- FILTROS - fundo #2c3e50 --- */
.barra-generos { display: flex; background-color: #2c3e50; }
.btn-genero {
  flex: 1; padding: 15px; background: none; border: none;
  border-right: 1px solid #444; color: #FFFFFF;
  font-size: 16px; cursor: pointer;
}
.btn-genero:last-child { border-right: none; }
.btn-genero:hover, .btn-genero.ativo { background-color: #1a5276; }

/* --- LISTA DE ITENS --- */
.grade-livros { padding: 20px; display: flex; flex-direction: column; gap: 15px; }
.ficha-livro { background: #FFFFFF; border: 1px solid #ddd; border-radius: 8px; padding: 15px; }
.titulo-livro { width: 100%; margin-bottom: 10px; text-align: center; font-weight: bold; }

/* --- INTERACOES --- */
.acoes-livro { display: flex; gap: 20px; align-items: center; }
.item-acao { display: flex; align-items: center; gap: 5px; cursor: pointer; }
.item-acao img { width: 24px; }

/* --- CAIXA DE COMENTARIO --- */
.caixa-resenha { width: 100%; margin-top: 15px; display: none; align-items: center; gap: 10px; }
.campo-resenha { flex: 1; padding: 10px; border: 1px solid #ccc; border-radius: 4px; }

/* --- PAGINACAO --- */
.navegacao { display: flex; justify-content: center; align-items: center; gap: 10px; padding: 20px; margin-top: auto; }
.btn-pagina { padding: 5px 15px; border: 1px solid #ccc; background: #FFFFFF; cursor: pointer; }
.btn-pagina:disabled { opacity: 0.5; cursor: not-allowed; }

/* --- MODAIS --- */
.overlay { display: none; position: fixed; top: 0; left: 0; width: 100%; height: 100%;
  background: rgba(0,0,0,0.5); justify-content: center; align-items: center; }
.overlay.ativo { display: flex; }
.dialogo { width: 400px; padding: 30px; background: #FFFFFF; border-radius: 8px; }
.topo-dialogo { display: flex; justify-content: space-between; align-items: center; margin-bottom: 20px; }
.btn-x { width: 20px; cursor: pointer; }

/* --- FORMULARIOS --- */
.campo { display: flex; flex-direction: column; margin-bottom: 15px; }
.linha-campos { display: flex; gap: 20px; }
.meia-largura { flex: 1; }
.campo label { margin-bottom: 5px; font-weight: 600; }
.campo input, .campo select {
  padding: 10px; border: 1px solid #ccc; border-radius: 4px;
}
.campo input.erro, .campo select.erro { border-color: #e74c3c; }

/* --- ACOES MODAIS --- */
.botoes-dialogo { display: flex; justify-content: flex-end; gap: 10px; margin-top: 20px; }
.btn-desistir { padding: 10px 20px; background: none; border: 1px solid #2c3e50;
  border-radius: 4px; color: #2c3e50; cursor: pointer; }
.btn-confirmar { padding: 10px 20px; background: #2c3e50; border: none;
  border-radius: 4px; color: #FFFFFF; cursor: pointer; }

```

Checkpoint Etapa 7: O visual corresponde as cores: sidebar `#2c3e50` , filtros `#2c3e50` / `#FFFFFF` , paginacao ativa `#1a5276` .

Etapa 8 — JavaScript da SPA (Foco: Modal de Login)

PARTE PRINCIPAL: O `app.js` faz todas as chamadas `fetch` a API e atualiza a tela sem recarregar (regra SPA). Abaixo, o código completo com foco no modal de login.

8.1 `app.js`

```
const API_URL = '/api';
let usuarioLogado = null;
let paginaAtual = 0;
let filtroAtual = null;
let totalPaginas = 1;

document.addEventListener('DOMContentLoaded', () => {
  const armazenado = localStorage.getItem('usuario');
  if (armazenado) {
    usuarioLogado = JSON.parse(armazenado);
    atualizarUIUsuarioLogado();
    buscarEstatisticasPerfil();
  }
  carregarItens();

  // Event Listeners
  document.getElementById('btn-sessao').addEventListener('click', tratarCliqueAutenticacao);
  document.getElementById('fechar-login').addEventListener('click', () => alternarModal('overlay-login', false));
  document.getElementById('btn-cancelar-entrada').addEventListener('click', () => alternarModal('overlay-login', false));
  document.getElementById('form-entrar').addEventListener('submit', tratarLogin);

  document.getElementById('btn-adicionar-livro').addEventListener('click', () => alternarModal('overlay-criar', true));
  document.getElementById('fechar-criar').addEventListener('click', () => alternarModal('overlay-criar', false));
  document.getElementById('form-novo-livro').addEventListener('submit', tratarCriar);

  document.getElementById('btn-retroceder').addEventListener('click', () => {
    if (paginaAtual > 0) { paginaAtual--; carregarItens(); }
  });
  document.getElementById('btn-avancar').addEventListener('click', () => {
    if (paginaAtual < totalPaginas - 1) { paginaAtual++; carregarItens(); }
  });

  // Filtros bloqueados se não logado
  document.querySelectorAll('.btn-genero').forEach(btn => {
    btn.addEventListener('click', (e) => {
      if (!usuarioLogado) { alternarModal('overlay-login', true); return; }
      const tipo = e.target.getAttribute('data-tipo');
      if (filtroAtual === tipo) { filtroAtual = null; e.target.classList.remove('ativo'); }
      else { filtroAtual = tipo;
        document.querySelectorAll('.btn-genero').forEach(b => b.classList.remove('ativo'));
        e.target.classList.add('ativo'); }
      paginaAtual = 0;
      carregarItens();
    });
  });

  // ===== FUNCAO ALTERNAR MODAL =====
  function alternarModal(idModal, mostrar) {
    document.getElementById(idModal).classList.toggle('ativo', mostrar);
  }

  // ===== FUNCAO TRATAR AUTENTICACAO =====
  function tratarCliqueAutenticacao() {
    if (usuarioLogado) {
      usuarioLogado = null;
      localStorage.removeItem('usuario');
      window.location.reload();
    } else {
      alternarModal('overlay-login', true);
    }
  }
}
```

```

// ===== FUNCAO LOGIN (MODAL) =====
async function tratarLogin(e) {
  e.preventDefault();
  const campoEmail = document.getElementById('campo-email');
  const campoSenha = document.getElementById('campo-senha');
  campoEmail.classList.remove('erro');
  campoSenha.classList.remove('erro');

  // Validacao: campos obrigatorios
  if (!campoEmail.value || !campoSenha.value) {
    if (!campoEmail.value) campoEmail.classList.add('erro');
    if (!campoSenha.value) campoSenha.classList.add('erro');
    alert('email ou senha obrigatorio');
    return;
  }

  try {
    const res = await fetch(`${API_URL}/login`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email: campoEmail.value, senha: campoSenha.value })
    });

    if (res.ok) {
      // Login bem-sucedido
      usuarioLogado = await res.json();
      localStorage.setItem('usuario', JSON.stringify(usuarioLogado));
      atualizarUIUsuarioLogado();
      alternarModal('overlay-login', false);
      buscarEstatisticasPerfil();
      carregarItens();
    } else {
      // Credenciais invalidas
      campoEmail.classList.add('erro');
      campoSenha.classList.add('erro');
      alert('email ou senha incorreta');
    }
  } catch (err) {
    console.error(err);
  }
}

// ===== ATUALIZAR UI APOS LOGIN =====
function atualizarUIUsuarioLogado() {
  const botao = document.getElementById('btn-sessao');
  botao.textContent = 'Logout';
  botao.classList.replace('btn-entrar', 'btn-sair');
  document.getElementById('nome-leitor').textContent = usuarioLogado.nomeUsuario;
  document.getElementById('foto-leitor').src = `assets/imagens_perfil/${usuarioLogado.imagem}`;
  document.getElementById('btn-adicionar-livro').disabled = false;
}

// ===== BUSCAR ESTATISTICAS DO PERFIL =====
async function buscarEstatisticasPerfil() {
  if (!usuarioLogado) return;
  const res = await fetch(`${API_URL}/perfil/${usuarioLogado.id}`);
  const stats = await res.json();
  document.getElementById('total-livros').textContent = stats.totalAtividades;
  document.getElementById('total-paginas').textContent = stats.totalCalorias;
}

// ===== CARREGAR ITENS COM PAGINACAO =====
async function carregarItens() {
  let url = `${API_URL}/livros?page=${paginaAtual}`;
  if (filtroAtual) url += `&tipo=${filtroAtual}`;
  try {
    const res = await fetch(url);
    const data = await res.json();
    totalPaginas = data.totalPages;
    renderizarItens(data.content);
    atualizarPaginacao();
  } catch (err) { console.error(err); }
}

// ===== RENDERIZAR ITENS =====
function renderizarItens(itens) {
  const lista = document.getElementById('lista-atividades');
  lista.innerHTML = '';
  itens.forEach(item => {
    const d = new Date(item.createdAt);

```



```

const dataFormatada =
  `${String(d.getHours()).padStart(2, '0')}:${String(d.getMinutes()).padStart(2, '0')} - ` +
  `${String(d.getDate()).padStart(2, '0')}/${String(d.getMonth() + 1).padStart(2, '0')}/${d.getFullYear()}`;

const jaReagiu = usuarioLogado && item.favoritos.some(r => r.membro.id === usuarioLogado.id);
const icone = jaReagiu ? 'CoracaoVermelho.svg' : 'coracao.svg';

const cartao = document.createElement('div');
cartao.className = 'ficha-livro';
cartao.innerHTML = `
  <div class="titulo-livro">
    <span style="text-transform:capitalize">${item.genero}</span>
    <span style="float:right;font-size:12px;font-weight:normal">${dataFormatada}</span>
  </div>
  <div class="dados-livro">
    
    <div>
      <strong>${item.membro.nomeUsuario}</strong><br>
      <strong>${item.titulo}</strong><br>
      ${item.numeroPaginas} páginas | Ano: ${item.anoPublicacao}
    </div>
    <div class="acoes-livro">
      <div class="item-acao" onclick="tratarReacao(${item.id})">
        
        <span>${item.favoritos.length}</span>
      </div>
      <div class="item-acao" onclick="alternarCaixaComentario(${item.id})">
        
        <span>${item.resenhas.length}</span>
      </div>
    </div>
    <div id="caixa-comentario-${item.id}" class="caixa-resenha">
      <input type="text" id="entrada-comentario-${item.id}" class="campo-resenha" placeholder="Escrever um comentario...">
      
    </div>
  </div>`;
  lista.appendChild(cartao);
});
}

// ===== ATUALIZAR PAGINACAO =====
function atualizarPaginacao() {
  document.getElementById('btn-retroceder').disabled = paginaAtual === 0;
  document.getElementById('btn-avancar').disabled = paginaAtual >= totalPaginas - 1;
  const numeros = document.getElementById('botoes-pagina');
  numeros.innerHTML = '';
  const botao = document.createElement('button');
  botao.textContent = paginaAtual + 1;
  botao.className = 'ativo';
  numeros.appendChild(botao);
}

// ===== REACAO (CURTIR/DESCURTIR) =====
async function tratarReacao(id) {
  if (!usuarioLogado) { alternarModal('overlay-login', true); return; }
  await fetch(`${API_URL}/livros/${id}/favoritar`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ usuarioId: usuarioLogado.id })
  });
  carregarItens();
}

// ===== ALTERNAR CAIXA DE COMENTARIO =====
function alternarCaixaComentario(id) {
  if (!usuarioLogado) { alternarModal('overlay-login', true); return; }
  const caixa = document.getElementById(`caixa-comentario-${id}`);
  caixa.style.display = caixa.style.display === 'flex' ? 'none' : 'flex';
}

// ===== ENVIAR COMENTARIO =====
async function enviarComentario(id) {
  const entrada = document.getElementById(`entrada-comentario-${id}`);
  const texto = entrada.value.trim();
  if (texto.length <= 2) {
    alert('nao e possivel enviar um comentario vazio');
    return;
  }
  await fetch(`${API_URL}/livros/${id}/resenhar`, {

```

```

    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ usuarioId: usuarioLogado.id, texto: texto })
  });
  carregarItens();
}

// ===== CRIAR NOVO ITEM =====
async function tratarCriar(e) {
  e.preventDefault();
  const tipo = document.getElementById('campo-genero');
  const distancia = document.getElementById('campo-paginas');
  const duracao = document.getElementById('campo-ano');
  [tipo, duracao].forEach(el => el.classList.remove('erro'));

  if (!tipo.value || !duracao.value) {
    if (!tipo.value) tipo.classList.add('erro');
    if (!duracao.value) duracao.classList.add('erro');
    alert('Campo obrigatorio');
    return;
  }

  await fetch(`${API_URL}/livros`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      usuarioId: usuariologado.id,
      genero: tipo.value,
      anoPublicacao: parseInt(distancia.value) || 0,
      numeroPaginas: parseInt(duracao.value),
      titulo: document.getElementById('campo-autor').value || 'Sem titulo',
      autor: document.getElementById('campo-autor').value || 'Desconhecido'
    })
  });
  alternarModal('overlay-criar', false);
  paginaAtual = 0;
  carregarItens();
  buscarEstatisticasPerfil();
}

```

Checkpoint Etapa 8: Sem logar: filtros/reacao/comentario abrem o modal de login. Logando com `saeplivros@email.com` / `123456` : a sidebar mostra o usuario, os totais carregam e as interacoes sao liberadas.

Etapa 9 — Checklist Final de Requisitos

#	Requisito	Status
1	Body dividido em colunas (sidebar + main)	[]
2	Sidebar com fundo #2c3e50 , fonte #FFFFFF	[]
3	Totais de Qtd. Livros Lidos e Qtd. Paginas vindos do banco	[]
4	Botao "Livro" desabilitado sem login, cor #FFFFFF	[]
5	Rodape com icones e "Copyright-2024"	[]
6	Botao Login no canto superior direito (bg #FFFFFF)	[]
7	Filtros Ficcao/Nao-Ficcao/Tecnico centralizados, fundo #2c3e50	[]
8	Cada item: titulo centralizado, dados do membro, data HH:MM - DD/MM/YY	[]
9	Icones coracao e comentario com contagem	[]
10	Paginacao limita 4 itens; pagina ativa com #1a5276	[]
11	Filtros, paginacao, reacao e comentario bloqueados sem login (abre modal)	[]
12	Modal login: campos email/senha com labels	[]
13	Botoes "Cancelar" (sem bg, borda #2c3e50) e "Login" (bg #2c3e50)	[]
14	Campo vazio: borda vermelha + mensagem de erro	[]
15	Credencial errada: borda vermelha + "email ou senha incorreta"	[]
16	Login correto: perfil atualiza com dados do membro	[]
17	Logout retorna a tela inicial	[]
18	Reacao toggle: clica, conta +1; clica de novo, -1	[]
19	Comentario > 2 caracteres; senao avisa "comentario vazio"	[]
20	Comentario e reacao persistidos no banco	[]
21	Cadastro de novo livro: campos obrigatorios validados	[]
22	Novo livro aparece no topo da lista	[]

Etapa 10 — Rodar e Testar End-to-End

10.1 Fluxo de Teste Completo

1. Pagina carrega com feed de 4 itens (paginacao) e sidebar do sistema.
2. Clicar em filtro/reacao sem login: abre modal de login.
3. Logar com `saeplivros@email.com` / `123456` : perfil muda, totais atualizam.
4. Reagir a um item: icone fica vermelho, contador +1; clicar de novo: -1.
5. Comentar com > 2 caracteres: comentario persiste.
6. Clicar em "Livro": modal de cadastro; preencher e salvar: aparece no topo.
7. Navegar entre paginas com Proxima/Anterior.
8. Logout: volta a tela inicial.

10.2 Estrutura Final de Arquivos

```
src/main/java/com/saep/saepbiblioteca/
├── SAEPBibliotecaApplication.java
├── model/
│   ├── Membro.java
│   ├── Livro.java
│   ├── Favorito.java
│   └── Resenha.java
├── repository/
│   ├── MembroRepository.java
│   ├── LivroRepository.java
│   ├── FavoritoRepository.java
│   └── ResenhaRepository.java
└── controller/
    └── ApiController.java

src/main/resources/
├── application.properties
├── data.sql
├── static/
│   ├── index.html
│   ├── style.css
│   ├── app.js
│   └── assets/ (icones, imagens_perfil, logo_*, ...)
```

10.3 Comandos para Executar

```
# 1. Garanta que o MySQL 8 esta rodando localmente (porta 3306)
# 2. Rodar a aplicacao
mvn spring-boot:run

# 3. Abrir no navegador
http://localhost:8080
```